WEEK 7 · LECTURE

Widgets, Layout & Navigation

Lifecycle · Keys · Row, Column & Stack · Lists · Themes · GoRouter

CS 442 | Mobile Application Development | Fall 2026

1

TODAY'S PLAN

90 Minutes, Seven Blocks

Warm-up and recap	5 min	slides 1-3
Lifecycle, BuildContext, three trees	14 min	slides 4-8
Composition and keys	10 min	slides 9-12
Layout: constraints, Row, Column, Stack	26 min	slides 13-23
Lists with ListView.builder	7 min	slides 24-25
Themes with Material 3	9 min	slides 26-29
Navigation and GoRouter	14 min	slides 30-35
Errors cheat sheet and recap	5 min	slides 36-37

2

WARM-UP · LAST WEEK

Three Quick Questions

1. Stateless or Stateful?
A Text that always says 'Hello'.

2. What does setState do?
Answer in one sentence.

3. Which runs once?
initState or build?

3

STATE LIFECYCLE · CLOCK EXAMPLE, PART 1 OF 3

Ticking Clock (1/3): The App

Dart

```dart
import 'dart:async';
import 'package:flutter/material.dart';

void main() => runApp(
      const MaterialApp(
        home: Scaffold(body: Center(child: Clock())),
      ),
    );

class Clock extends StatefulWidget {
  const Clock({super.key});
  @override
  State<Clock> createState() => _ClockState();
}
```

import 'dart:async'
gives us Timer

main()
shows Clock in the middle of the screen

Clock
the widget; createState() makes its State

4

STATE LIFECYCLE · CLOCK EXAMPLE, PART 2 OF 3

Ticking Clock (2/3): initState

Dart

```dart
class _ClockState extends State<Clock> {
  late Timer _timer;

  @override
  void initState() {
    super.initState();
    _timer = Timer.periodic(
      const Duration(seconds: 1),
      (_) => setState(() {}),
    );
  }
  // dispose() and build() are on the next slide
```

late Timer _timer
a promise: set before it is used

initState
runs once: start the timer here

Every second
setState(() {}) asks for a rebuild

5

STATE LIFECYCLE · CLOCK EXAMPLE, PART 3 OF 3

Ticking Clock (3/3): dispose & build

Dart

```dart
  // ...still inside _ClockState
  @override
  void dispose() {
    _timer.cancel();    // stop the timer
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final t = DateTime.now().toString();
    return Text(t.substring(11, 19),
        style: const TextStyle(fontSize: 48));
  }
}
```

dispose
runs once at the end: cancel the timer

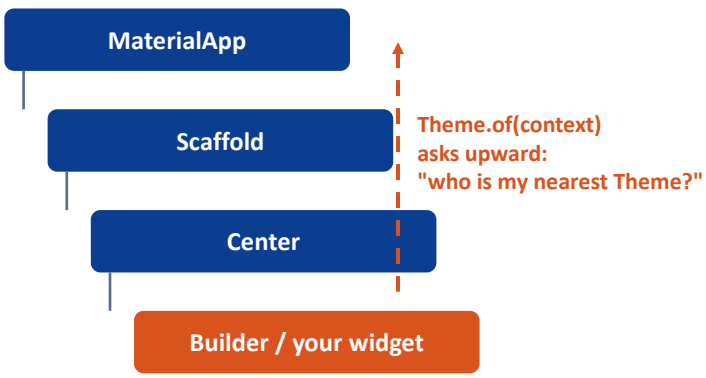
build
runs every second: reads the time

substring(11, 19)
keeps only HH:MM:SS

6

FINDING YOUR PLACE

BuildContext: Where Am I in the Tree?



context =
your address in the widget tree

of(context)
walks UP from you to find a parent

Needs a parent
no MaterialApp above = error

7

UNDER THE HOOD

Three Trees: Widget, Element, RenderObject

	Job	Lives
Widget	**The description** Immutable and cheap. Thrown away and rebuilt all the time.	Short
creates		
Element	**The live instance** Sits at one spot in the tree. Holds the State. Decides what to reuse.	Long
creates		
RenderObject	**The worker** Does the real layout and painting on the screen.	Long

When you call setState, new Widgets are built. The Elements stay, and compare old and new.

8

FLUTTER'S DESIGN STYLE

Composition Over Inheritance

Dart

```dart
// Inheritance: a new class for every look
class BigRedText extends Text { ... }
class BigBlueText extends Text { ... }

// Composition: small widgets wrapped together
Center(
  child: Text('Hello'),
)
```

Inheritance
one class per variation: explodes

Composition
wrap small widgets, like Lego

Center is a widget
even centring is a widget

9

KEYS · PART 1 OF 3

Why Does super.key Exist?

Before
A tapped 3 times

A: 3 B: 0

Swap, no keys
counts stay in place

B: 3 A: 0

Swap, with keys
state moves with its widget

B: 0 A: 3

Flutter matches by
type + position, unless a key says otherwise

Key = name tag
ValueKey('A') identifies a widget

super.key
passes that tag up to Widget

10

KEYS · PART 2 OF 3

A Tile That Remembers Its Taps

Dart

```dart
class Tile extends StatefulWidget {
  const Tile(this.label, {super.key});
  final String label;
  @override
  State<Tile> createState() => _TileState();
}
class _TileState extends State<Tile> {
  int taps = 0;   // lives in the State
  @override
  Widget build(BuildContext context) =>
      ElevatedButton(
        onPressed: () => setState(() => taps++),
        child: Text('${widget.label}: $taps'));
}
```

taps
the memory: lives in State

widget.label
reads the widget's field

{super.key}
optional name tag

KEYS · PART 3 OF 3

Swap the Tiles: With and Without Keys

Dart

```dart
class _SwapState extends State<Swap> {
  var tiles = [
    Tile('A', key: ValueKey('A')),
    Tile('B', key: ValueKey('B')),
  ];
  void swap() => setState(
      () => tiles = tiles.reversed.toList());
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Row(children: tiles),
      ElevatedButton(onPressed: swap, child: Text('Go')),
    ]);
  }
}
```

Tap A three times
then press Go

With keys
A:3 moves to the right

Remove the keys
the 3 stays on the left

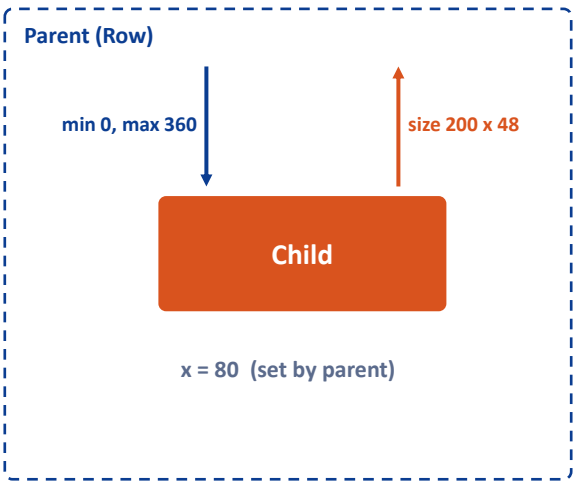
LAYOUT · THE ONE RULE

Constraints Go Down, Sizes Go Up

1 **Parent gives constraints**
"You may be 0 to 360 wide."

2 **Child picks a size**
"Then I will be 200 wide."

3 **Parent sets position**
"I will put you at x = 80."



LAYOUT · ROW AND COLUMN

Row and Column: Widgets in a Line

Dart

```dart
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('Pizza'),
    Text('Fresh and hot'),
    Row(children: [
      Icon(Icons.star),
      Text('4.8'),
    ]),
  ],
)
```

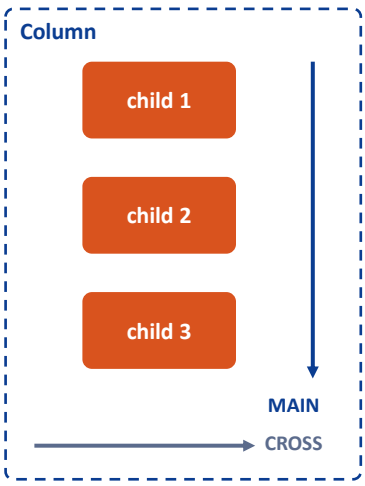
Column
children go top to bottom

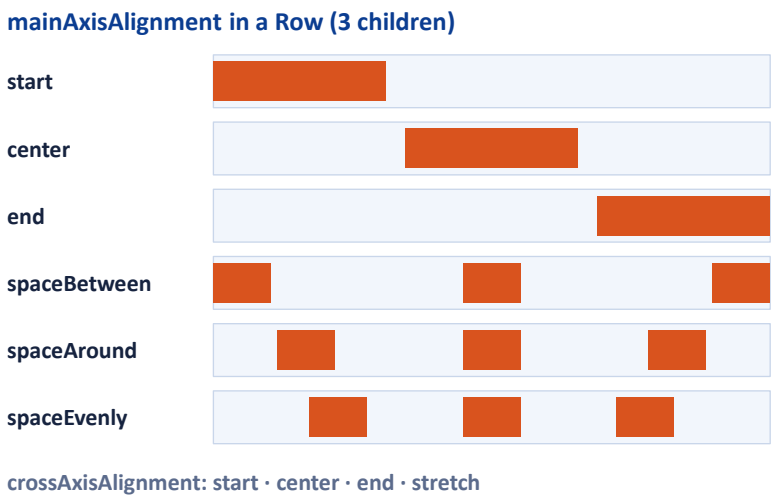
Row
children go left to right

children: [ ]
a list of widgets

LAYOUT · AXES

Main Axis vs Cross Axis





15

LAYOUT · WHEN IT DOES NOT FIT

The Yellow-Black Stripes: Overflow



Row asked for more width than the screen allows.

Four ways to fix it

Expanded
share the space; text wraps

Wrap
moves to the next line

Scroll
ListView or SingleChildScrollView

Shrink
smaller font or fixed sizes

16

LAYOUT · SHARING SPACE

Expanded and Flexible: Dividing the Row

Row



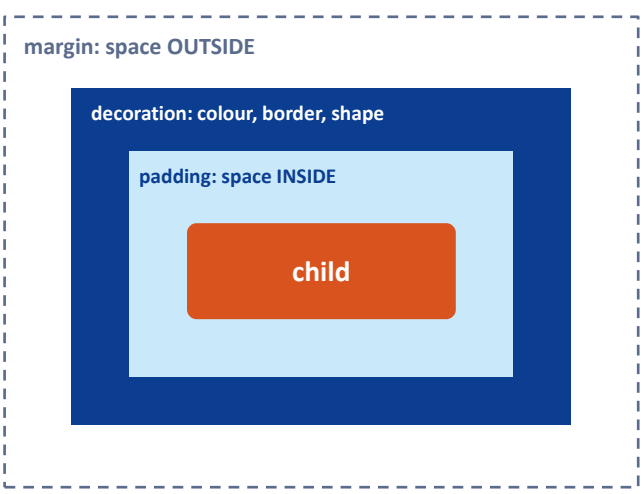
The fixed box takes what it asks for. The rest is shared 1 : 2.

Expanded
Child MUST fill its share.
flex says how big a share.

Flexible
Child MAY be smaller than its share.
Expanded = Flexible with fit: tight.

LAYOUT · BOXES AND SPACING

Container: Margin, Border, Padding



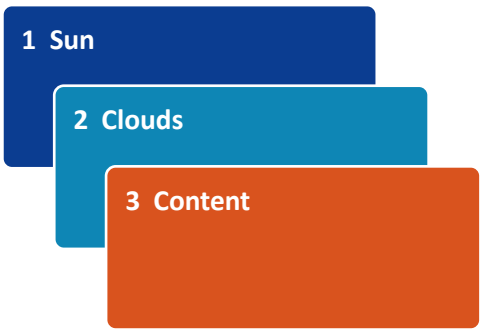
margin
outside the box, pushes neighbours away

padding
inside the box, pushes the child away

Only need space?
Use Padding or SizedBox. Lighter.

LAYOUT · STACK

Stack: Widgets on Top of Each Other



first child = bottom layer

```
Stack(
  children: [
    Sun(),
    Positioned(
      top: 20, right: 16,
      child: Clouds(),
    ),
    Content(),
  ],
)
```

Positioned
top, left, right, bottom: pin to an edge